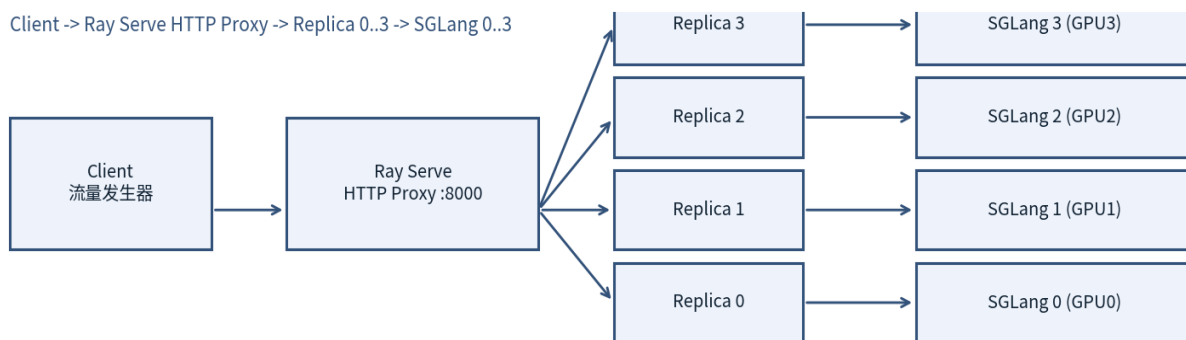


HW3 挑战三：比较并改进 Ray Serve 路由

附：目标一 前缀缓存测量对照 · 学号 0102603133 · 4 × RTX 4090D

1. 实验环境与方法

硬件为 AutoDL 单节点 4 × NVIDIA GeForce RTX 4090 D (24 GB)，驱动 580.105.08、CUDA 13.0，256 vCPU / 755 GB 内存。软件为 SGLang 0.5.14 (torch 2.11.0+cu130、transformers 5.8.1) 与 Ray 2.56.0，两者位于独立 Python 环境、仅通过 HTTP 通信；模型统一为 Qwen/Qwen3-0.6B，RadixCache 全程开启。目标三使用课程固定负载 mooncake_prefix_workload_v2_seed2026.jsonl (sha256 f154ba95...75b03，64 前缀族 / 64 预热 / 2048 测量，steady 40 req/s、burst 120 req/s、recovery 60 req/s)，A-D 五轮均通过同一个 Ray Serve HTTP 入口回放，每轮先重部署、再对四个后端 POST /flush_cache 确认成功。

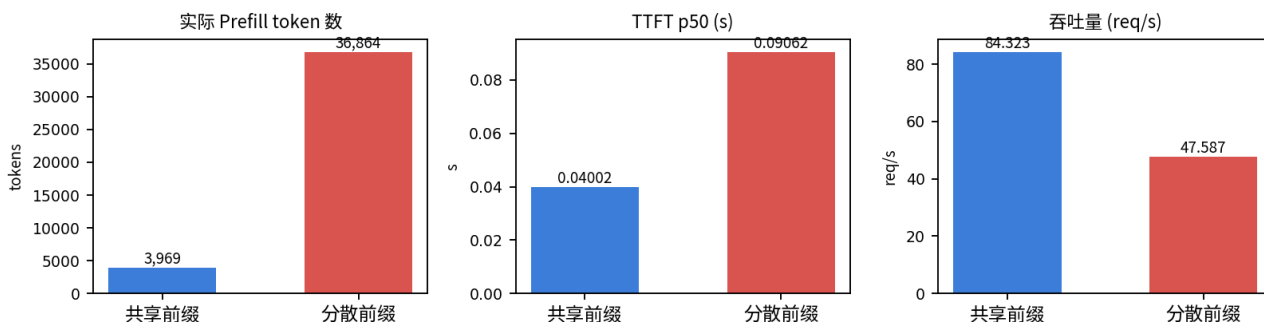


拓扑：Client → Ray Serve HTTP Proxy(:8000) → 4 个 Replica (各自落在独立逻辑 worker 节点) → 4 个独立 SGLang 后端 (GPU 0-3，端口 31000-31003)。每轮均为 1 个 head + 4 个 worker 逻辑节点，由 ray.cluster_utils.Cluster 创建，Replica 经 replica_slot 资源约束绑定到各自节点，并按节点 ID 顺序固定转发到一个后端。

2. 目标一：前缀缓存测量 (SGLang RadixCache)

两组各 32 条测量请求，最大并发 8，均走原生 /generate 接口发送 input_ids 并使用流式响应，统一 temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026。两条请求序列长度完全相同 (1152 tokens = 1024 前缀 + 128 后缀)：共享前缀组 32 条请求共用同一段 1024-token 前缀；分散前缀组每条请求的前缀彼此无关。每组测量前等待在飞请求结束并确认 /flush_cache 成功，共享前缀组额外先发送一条预热请求使公共前缀进入 RadixCache；预热请求不计入结果。

指标	共享前缀组	分散前缀组	差异
成功请求 / 总数	32/32	32/32	一致
缓存命中率	89.23%	0.00%	+89.23 pp
命中 token 数	32,895	0	+32,895
实际 Prefill token 数	3,969	36,864	-32,895 (-89.2%)
吞吐量 (req/s)	84.32	47.59	+77.2%
TTFT p50 / p95 (s)	0.0400 / 0.0568	0.0906 / 0.1103	p50 -55.8%
TPOT p50 / p95 (s)	0.0032 / 0.0038	0.0054 / 0.0079	p50 -41.1%
端到端 p50 / p95 (s)	0.0922 / 0.1040	0.1633 / 0.1761	p50 -43.5%



为什么前缀可以复用：从因果自注意力看

自注意力为 $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$ 。因果解码器中，位置 t 的查询 q_t 只与位置 $\leq t$ 的 K 、 V 发生作用 (因果掩码把 $j > t$ 的分数置为 $-\infty$)。因此一段前缀 $p_{1..m}$ 一旦确定，它对应的 $K_{1..m}$ 、 $V_{1..m}$ 就与后续生成内容无关：权重矩阵是逐层固定的，位置编码只依赖绝对/相对位置而不依赖同批次里的其它请求。于是在相同

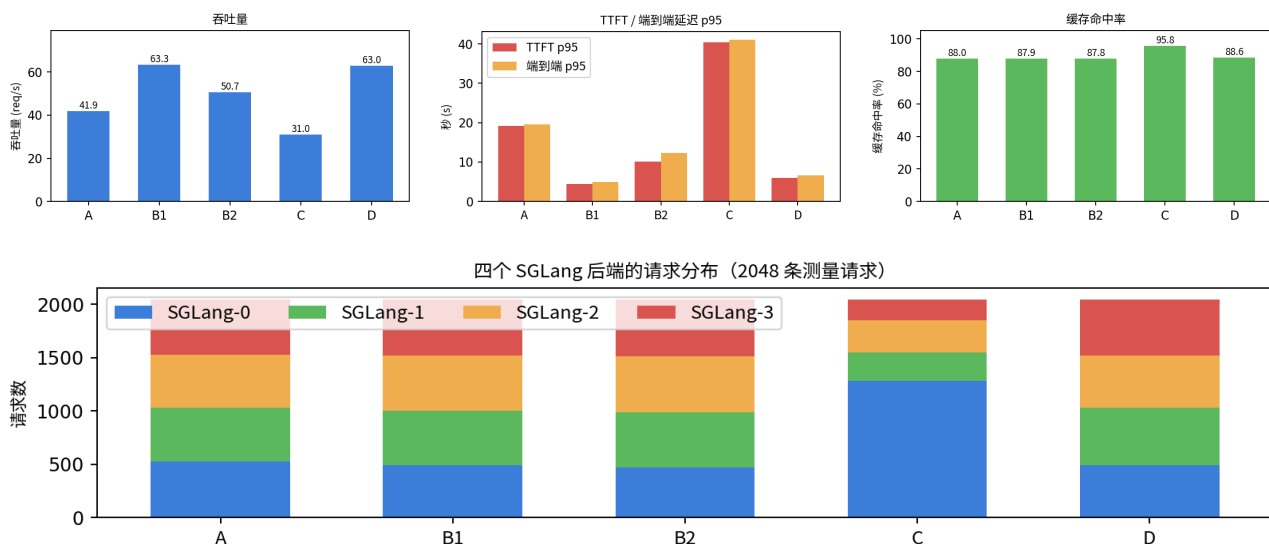
token 与相同位置上，前缀的 K/V 可以被任何以该前缀开头的请求复用——RadixCache 正是按 token 前缀把 K/V 组织成基数树并共享这些节点。

命中后仍需计算的部分只有未命中的那一段：本实验共享前缀组每条 1152 tokens 中约 1024 tokens 命中（实测命中率 89.23%，因为每条请求各自的 128-token 后缀必须新算），实际 prefill 从 36 864 降到 3 969 tokens。TTFT 因此大幅下降：它主要由 prefill 计算量决定，前缀命中直接削减了这条路径。TPOT 也下降了（p50 0.0054 → 0.0032 s，-41.1%），但这是间接收益：缓存并不减少 decode 的计算量，而是共享前缀组插入的 prefill 批更短、对 decode 的抢占更少，decode 步更连续，因此降幅仍小于 TTFT（-55.8%）。端到端延迟由 TTFT 与 decode 时间共同组成，其改善（-43.5%）介于两者之间。

3. 目标三：四组路由对照

组	路由器	并发上限	成功	吞吐 (req/s)	缓存命中率	实际 Prefill token	TTFT p50/p95 (s)	端到端 p95 (s)	耗时 (s)	四后端请求分布 SGLang-0/1/2/3
A	p2c	5	2048	41.94	88.02%	387,453	7.48/19.19	19.56	48.8	524/506/500/518
B1	p2c	16	2048	63.31	87.89%	391,424	1.68/4.37	4.99	32.3	488/511/522/527
B2	p2c	32	2048	50.69	87.83%	393,597	3.96/10.13	12.30	40.4	470/514/527/537
C	consistent_hash	16	2048	30.98	95.78%	136,444	1.35/40.38	41.04	66.1	1280/267/303/198
D	affinity_load	16	2048	62.99	88.59%	369,024	1.28/5.93	6.63	32.5	492/539/490/527

五组均满足每轮 2048 条测量请求成功且完整返回（failed=0、incomplete=0、warmup 64/64、validation_errors 为空），四个 Replica / 四个节点 / 四个后端均被覆盖；dispatch_lag_p95 ≤ 0.0018 s、client_queue_p95 ≈ 0.0003 s，说明压测端没有成为瓶颈。



3.1 默认值与 B 的选定值（max_ongoing_requests 5 → 16 → 32）

A 组用默认 p2c 与默认准入 5，四个副本合计只允许 20 条请求同时在飞。负载的到达速率为 40/120/60 req/s，远超该容量，请求只能在 Serve 侧排队：TTFT p95 19.19 s、端到端 p95 19.56 s、吞吐仅 41.94 req/s，回放耗时 48.8 s（计划 29.5 s）。放宽到 16（B1）后排队显著缓解，吞吐升到 63.31 req/s（+51%）、TTFT p95 降到 4.37 s（-77%），耗时 32.3 s 已接近计划值。继续放宽到 32（B2）反而退化：吞吐降到 50.69 req/s、TTFT p95 回升到 10.13 s，TPOT p95 从 0.0057 s 恶化到 0.0185 s（约 3.3 倍）——单副本接纳过多请求会拉大 decode 批，每个请求的每 token 时间被摊薄到更长，队头阻塞重新出现。因此 B 选定 16，C、D 均沿用该值。

3.2 前缀亲和：缓存收益与热点（C 组）

C 组使用 ConsistentHashRouter 且 num_fallback_replicas=0（严格按 X-Session-Id 哈希、无回退），同一前缀族恒定落在同一副本，于是缓存命中率升到 95.78%（B1 为 87.89%），实际 prefill token 从 391 424 降到 136 444（-65.1%），这是前缀亲和的直接收益。代价是负载按族而非按请求量分布：负载含一个超热前缀族（768 条测量请求、每条 190 tokens），它连同哈希到同一副本的其它族一起，让 SGLang-0 承担 1280/2048（62.5%），其余三个副本只有 198–303 条。热点副本成为系统瓶颈，整体吞吐跌到 30.98 req/s（B1 的 49%），TTFT p95 40.38 s、端到端 p95 41.04 s，分阶段看 steady 11.41 s、burst 42.34 s、recovery 37.79 s——缓存省下的 prefill 时间，被热点副本的排队时间全部吞掉还倒亏。这正说明：缓存收益是每副本局部的，而吞吐由最慢的那个副本决定。

3.3 组 D：前缀亲和优先 + 负载感知降级

D 组自研路由器

AffinityLoadAwareRouter (src/target3/routers.py) 只使用当前请求与当前服务状态：信号为（1）请求的 X-Session-Id（等于前缀族 ID），（2）各副本在飞请求数（由路由器生命周期钩子 on_request_routed /

on_request_completed 维护，并以副本 record_routing_stats 上报值兜底），（3）副本的 max_ongoing_requests。**规则为：**先用 blake2b 对 session id 做稳定哈希得到首选副本；若首选副本在飞数 $< 0.5 \times$ 上限，就只把请求给它，保持严格前缀亲和；一旦首选副本进入饱和（ $\geq 0.5 \times$ 上限），就把「当前最空闲副本 + 首选副本」放进同一优先级交给 Serve 探测两者队列长度择优，热点请求自然溢出到冷副本，且请求永远有落脚点。路由器不读取任何未来请求，也不修改负载。

结果：D 在保持均衡分布（490/539/490/527，与 B1 相当）的同时把缓存命中率从 B1 的 87.89% 提到 88.59%，实际 prefill token 从 391 424 降到 369 024（-5.7%）；吞吐 62.99 req/s 与 B1（63.31）基本持平，TTFT p95 5.93 s、端到端 p95 6.63 s。相对 C 组，D 用 7.2 pp 的命中率（95.78% $\rightarrow$ 88.59%）和 2.7 倍的 prefill token（136 444 $\rightarrow$ 369 024）换来了 2.03 倍吞吐（30.98 $\rightarrow$ 62.99 req/s）和 85.3% 的 TTFT p95 下降（40.38 $\rightarrow$ 5.93 s）。分阶段看，D 的 steady 阶段几乎无排队（TTFT p95 0.07 s），burst 阶段 6.7 s、recovery 5.67 s，说明溢出只在真正需要的时刻发生。

付出的代价与工程教训：（1）缓存局部性：溢出请求落到冷副本要重新 prefill 整段前缀，并会把该副本 RadixCache 中其它族的节点挤走，因此 D 的命中率明显低于严格亲和的 C；（2）实现成本：D 依赖 Ray 私有 API（ray.serve._private.request_router），版本升级可能失效。调试过程中出现过一次严重故障：自定义路由器若只继承 RequestRouter 而不带 FIFOMixin，会丢失 Ray 内置的 FIFO 排队语义，在 2048 条高并发请求下路由任务退化，实测 405/2048 请求失败、吞吐跌到 5 req/s；补上 FIFOMixin 后即恢复（该过程保留在 README 的排错记录中）。

4. 结论

（1）前缀缓存的收益在 token 维度上是真实的：目标一中共享前缀把实际 prefill 削减 89.2%、TTFT p50 降低 55.8%；但它只影响 prefill 路径，TPOT 与端到端延迟的改善幅度更小。（2）在四副本服务里，纯缓存视角的最优（严格亲和，C 组）会因为负载倾斜而在系统层面变成最差：命中率最高却吞吐最低、延迟最高。（3）准入上限是最便宜也最有效的旋钮：默认 5 明显过小，16 为本次负载的最优点，32 因 decode 批过大而回退。（4）组 D 表明，只需要「当前请求 + 当前在飞数」两个本地信号，就能在几乎不损失吞吐的前提下把命中率抬高于纯负载均衡，并把极端热点下的 p95 尾延迟压低一个数量级。后续可改进的方向是把负载信号换成副本上报的真实队列长度，并用更细的阈值自适应调节溢出比例。

复现方式：见 README.md。报告中所有数字均可在 results/target1/{shared_prefix,dispersed_prefix}/summary.json 与 results/target3/*/summary.json、main_table.csv 中逐一对齐。